

DVWA Security Level Comparison – SQL Injection & Cross-Site Scripting

Objective

The objective of this project is to compare how DVWA behaves at different security levels (Low, Medium, High). Two common web vulnerabilities – SQL Injection and Cross-Site Scripting (XSS), are used to identify the defense mechanisms implemented at each level.

Environment Setup:

- **Platform:** Kali Linux
- **Application:** DVWA (Damn Vulnerable Web App)
- **Start DVWA:**

sudo dvwa-start

- **Login Credentials:**
 1. Username: admin
 2. Password: password

Methodology:

1. SQL Injection

- **Low Security**
 - DVWA Security → Set Security to **Low**

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

Low

Security level set to low

➤ Open **SQL Injection** page.

Payload: **1' OR '1'='1**

Result:

Vulnerability: SQL Injection

User ID:

ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith

- **Medium Security**
→ Changed Security level to **Medium**.

DVWA Security

Security Level

Security level is currently: **medium**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

Medium

Security level set to medium

Result:

Vulnerability: SQL Injection

User ID:

ID: 4
First name: Pablo
Surname: Picasso

Http Capture:

```
Request  Response
Pretty  Raw    Hex
1 POST /vulnerabilities/sqli/ HTTP/1.1
2 Host: 127.0.0.1:42001
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 18
9 Origin: http://127.0.0.1:42001
10 Connection: keep-alive
11 Referer: http://127.0.0.1:42001/vulnerabilities/sqli/
12 Cookie: PHPSESSID=5fd9adb51786e5c4ccd9972a6c016c4c; security=medium
13 Upgrade-Insecure-Requests: 1
14 Sec-Fetch-Dest: document
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-User: ?1
18 DNT: 1
19 Sec-GPC: 1
20 Priority: u=0, i
21
22 id=1&Submit=Submit
```

- **High Security:**

→ Changed Security Level to **High**.

DVWA Security

Security Level

Security level is currently: **high**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

High

▼

Submit

Security level set to high

Result:

Vulnerability: SQL Injection

Click [here to change your ID.](#)

- **Went inside the link.**
- **Payload Used:** ' OR '1'='1

Vulnerability: SQL Injection

Click [here to change your ID.](#)

ID: ' OR '1'='1
First name: admin
Surname: admin

2. CROSS-SITE SCRIPTING (Reflected XSS)

- **Low Security**
 - Set security level to **Low**

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

Low

Security level set to low

→ XSS(Reflected) Page → Entered the payload.

Payload:

`<script>alert('XSS')</script>`

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Result:



- **Medium Security**

→ Set Security Level to **Medium**.

DVWA Security

Security Level

Security level is currently: **medium**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Medium

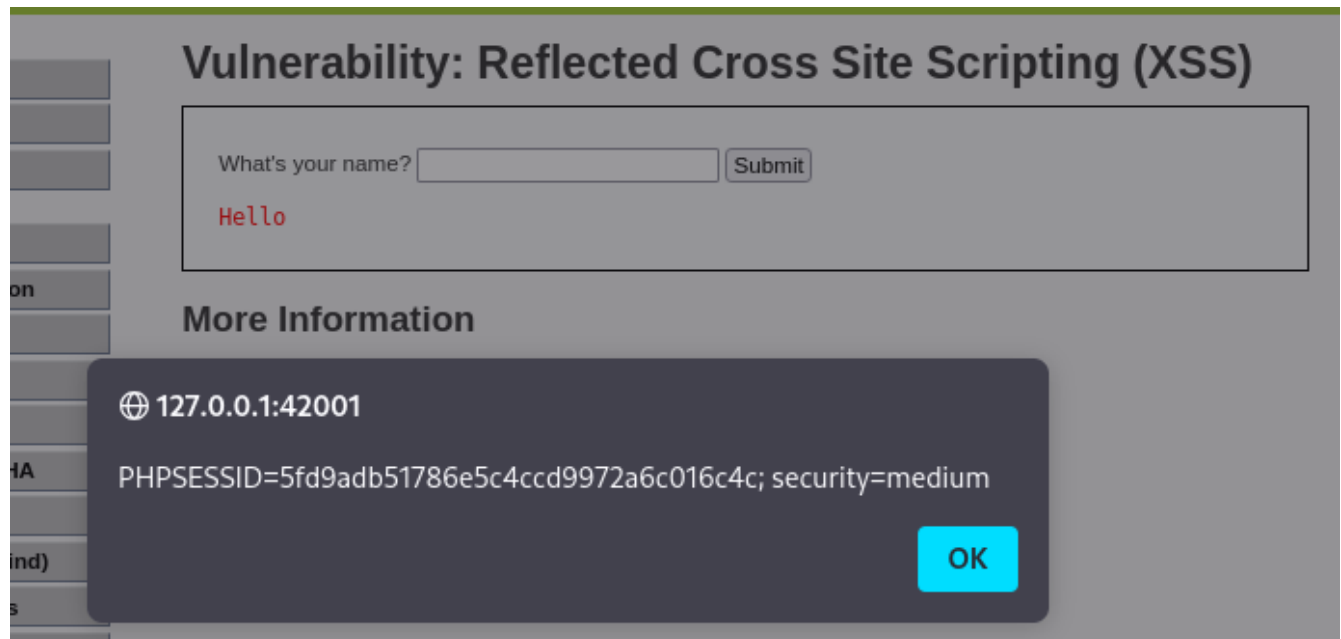
Security level set to medium

Payload Used:

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Result:



- **High Security:**

→ Changed security level to **High**.

DVWA Security

Security Level

Security level is currently: **high**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

High  Submit

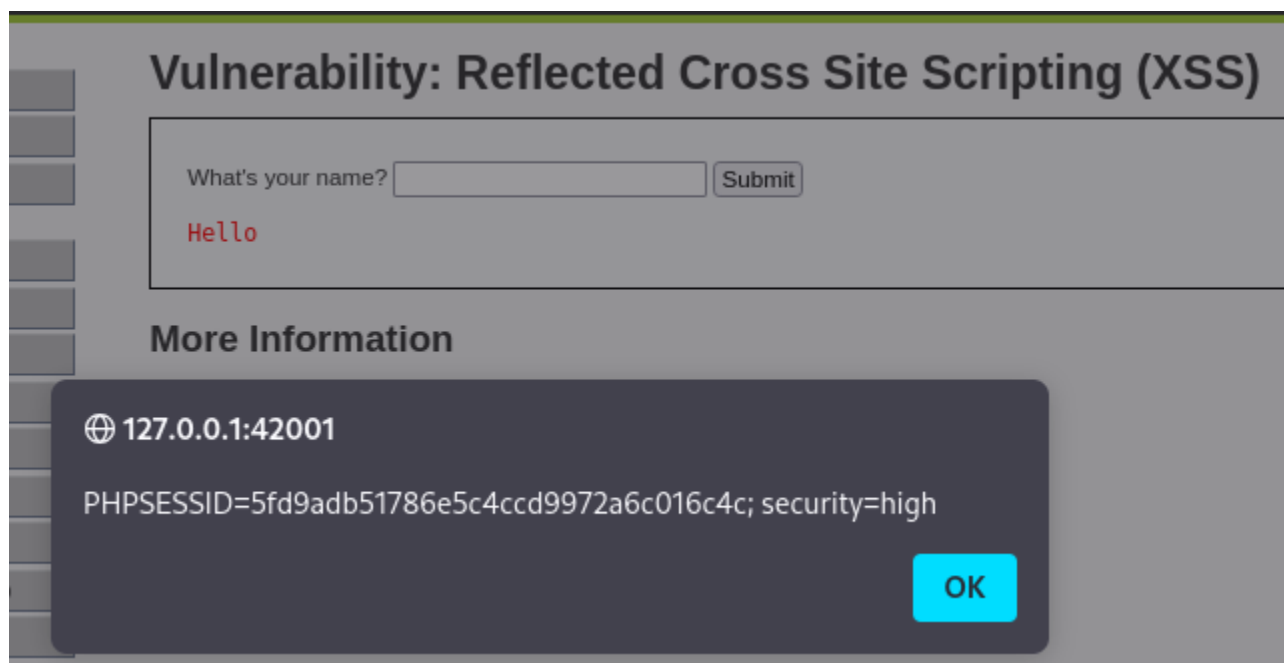
Security level set to high

Payload:

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name? Submit

Result:



Comparison:

SQL Injection

- **Low** – Direct SQLi works with 'OR '1'='1
- **Medium** – Input field removed. Was only able to get the users name one by one.
- **High** – Parametered queries /stronger input validation prevents SQLi by a higher level.

XSS (Reflected)

- **Low** - <script> payload executes directly.
- **Medium** - <script> filtered. Bypass possible with .
- **High** – Stricter filtering / encoding stops direct scripts but some HTML payloads may still fire.

Observation:

- **Medium Security:** Basic input filtering or restricting user input mechanisms (drop-down instead of free test).
- **High Security:** Parameterized queries, server-side validation, and stronger sanitation/encoding to neutralize malicious input.

Conclusion:

DVWA progressively implements stronger defenses as the security level increases. At **Low**, vulnerabilities are fully exploitable. At **Medium**, superficial defenses like input restrictions and script filtering make attacks harder but not impossible. At **High**, parameterized queries and stricter sensitization significantly reduce exploitable vulnerabilities. This exercise demonstrates how incremental defensive measures impact common web application attacks.